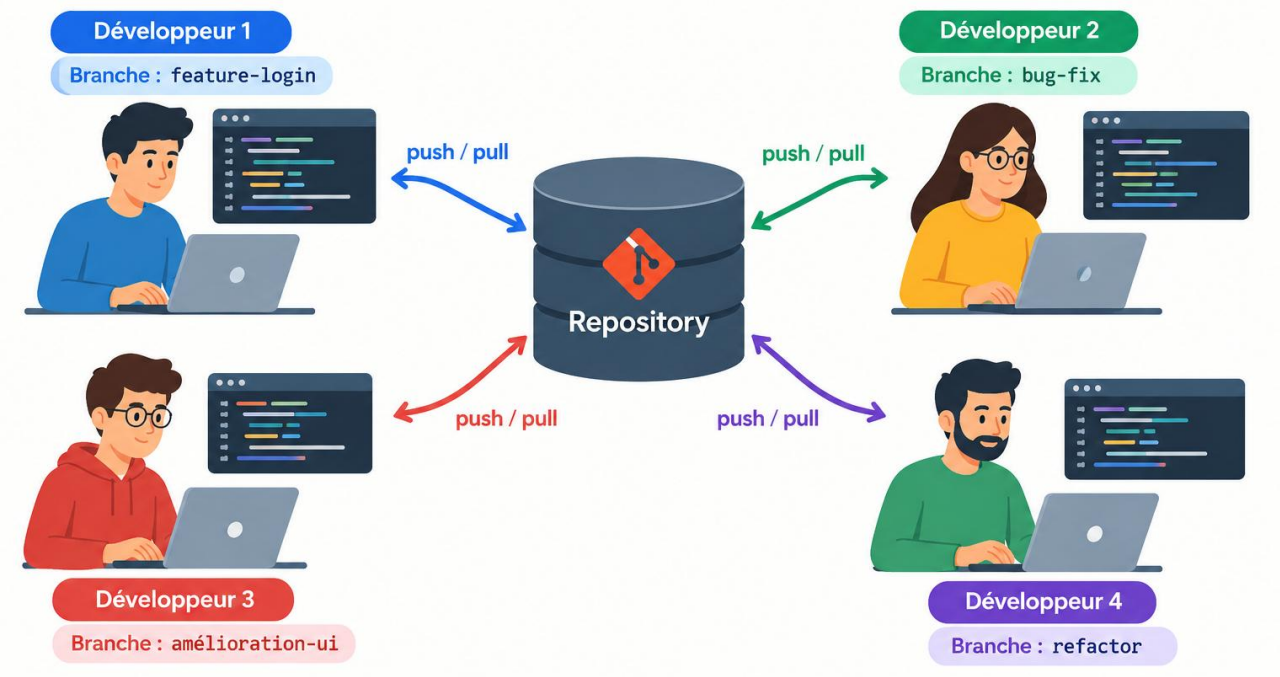
Système de contrôle de version

Abdelkader Ouared

abdelkader.ouared@univ-tiaret.dz

Système de contrôle de version (VCS: Version Control System)

- Compétence recherchée en entretien d'embauche
- GitHub : le CV des informaticiens
- **Continuous Security for GitOps**: Sécurité continue dans GitOps



Comment gagner en visibilité grâce à la cybersécurité sur GitHub ?

Objectif : construire un portfolio de cybersécurité crédible et valoriser ses compétences auprès des recruteurs.

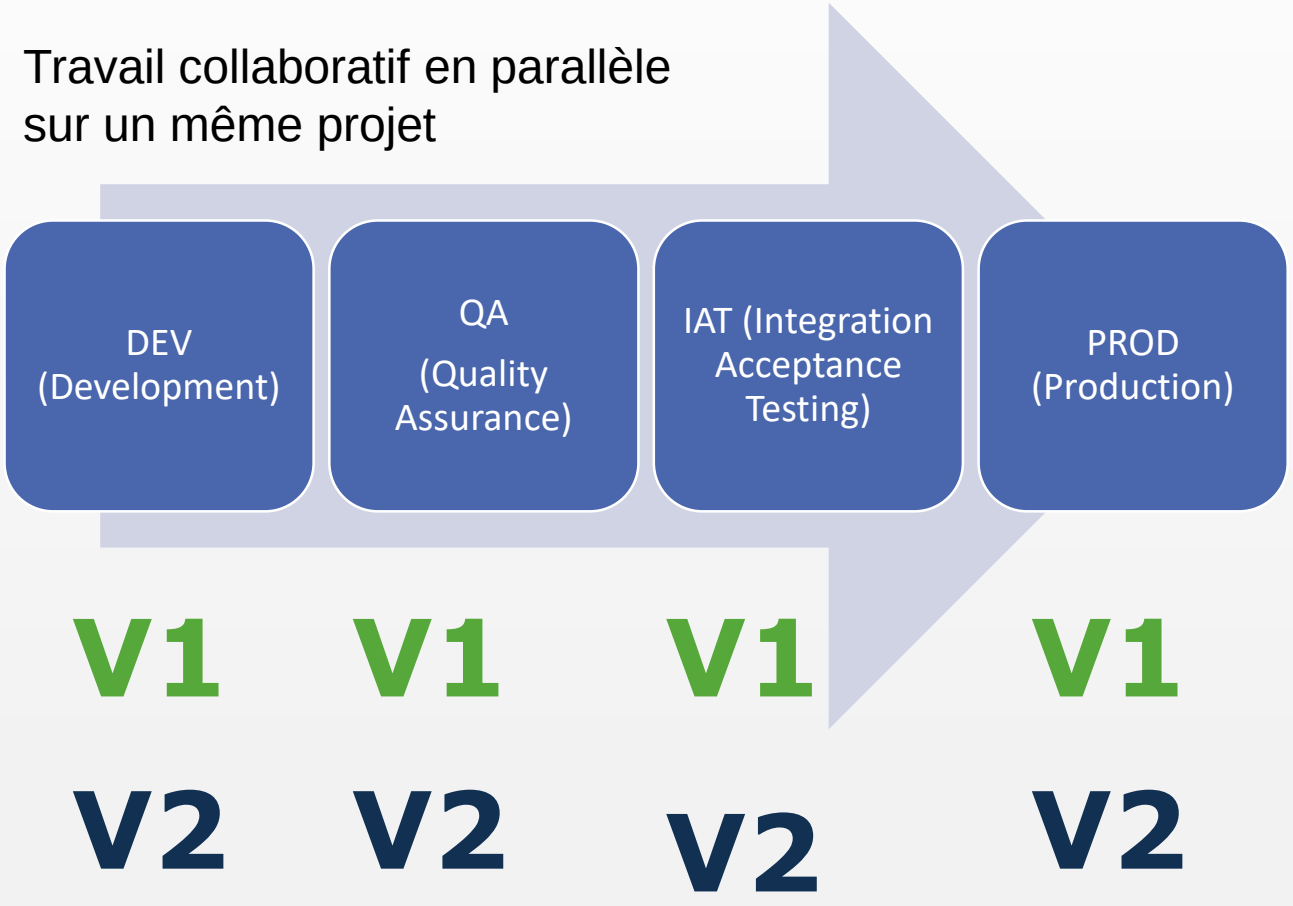
- **Détecter** : utiliser CodeQL (codeql.github.com) pour rechercher des vulnérabilités dans le code.
- **Explorer** : découvrir les projets de sécurité OWASP (Open Worldwide Application Security Project).
- **Contribuer** : corriger des bugs et proposer des *Pull Requests*.
- **Partager** : publier ses projets et documenter ses analyses de sécurité.
- **Se distinguer** : participer à des programmes Bug Bounty autorisés.

- **OWASP:** Organisation à but non lucratif qui propose des guides, des outils et des projets pour améliorer la sécurité des applications (https://github.com/OWASP?utm_source=chatgpt.com).
- **Bug Bounty** (programme de récompenses financières pour la découverte et le signalement responsable de vulnérabilités de sécurité, dans le respect des règles définies).

Exemple de Motivation

Exemple de Motivation

Travail collaboratif en parallèle sur un même projet



V1 — VERSION STABLE

L'application démarre.
Les tests passent.
Les utilisateurs travaillent.

↓

V2 — MODIFICATION

Ajout d'une fonctionnalité.
Modification de plusieurs fichiers.

↓

INCIDENT

Une erreur apparaît.

- Le paiement ou la connexion ne fonctionne plus.

Un correctif ou une nouvelle fonctionnalité peut introduire une régression.

Exemple de Motivation

Problèmes….. liés à la version V2



Rollback ????

Méthode manuelle: : une collaboration difficile

- Télécharger le projet
- Modifier des fichiers localement
- Envoyer les fichiers par e-mail ou messagerie
- Copier/coller les changements

Risques : doublons, écrasement du travail, versions confuses.

Question: si V2 casse l'application, comment retrouver exactement le code de V1 ?

Sans historique fiable, il faut chercher manuellement, reconstruire les fichiers ou recommencer.

Les copies manuelles ne suffisent pas

Dossier A

app.py
ancienne version

Dossier B

app_final.py
modification inconnue

Dossier C

app_final_vraiment_final.py
version supposée stable

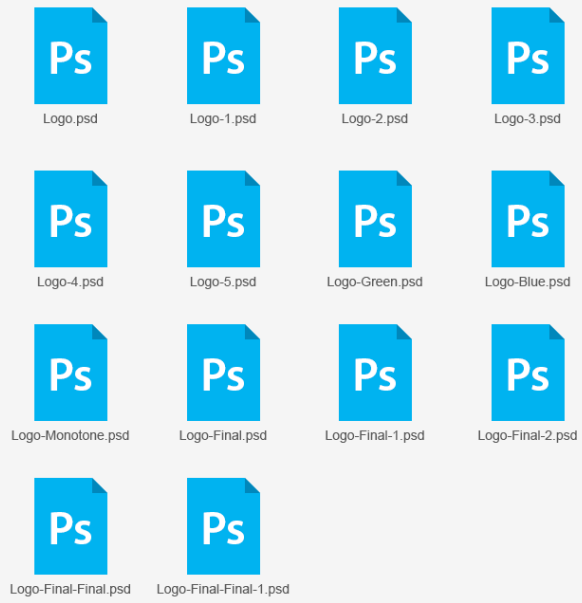
Problèmes :

Quelle copie est réellement la bonne ?

Quelles lignes ont changé ?

Qui a fait la modification ?

Comment fusionner le travail de deux développeurs sans perdre leurs changements ?



Sans contrôle de version ….

Modifications difficiles à annuler

Un fichier sauvegardé remplace la version précédente si aucune copie fiable n’existe.

Aucune traçabilité

Impossible d’identifier facilement l’auteur, la date et la raison d’un changement.

Échec de livraison

Une régression peut être déployée en QA, IAT ou production.

Interruption de service

L’application peut devenir indisponible jusqu’à la correction.

Système de contrôle de version

Pourquoi est-il indispensable dans un projet logiciel ?

- Un code qui fonctionne aujourd’hui peut être cassé demain.
- Comment retrouver une version stable, comprendre les changements et travailler en équipe ?

TRACER

Qui a changé quoi, et quand ?

COMPARER

Qu’est-ce qui a changé entre V1 et V2 ?

REVENIR

Comment restaurer une version stable ?

Sans VCS Version Control System : une collaboration difficile

Avec un VCS

- Chaque contribution est enregistrée
- Les développeurs travaillent en parallèle
- Les changements sont comparés et fusionnés
- L’historique permet de revenir à une version antérieure

Objectif : partager le travail sans perdre les contributions ni écraser les fichiers des autres.

Système De Contrôle De Version (VCS: Version Control System)

- **Un outil qui enregistre l'historique des modifications (Backup) d'un projet et permet de gérer ses différentes versions.**
- **Un VCS enregistre et suit les modifications apportées aux fichiers au fil du temps.**

Conserver l'historique

Identifier les changements et consulter les versions précédentes.

Comparer

Voir ce qui a été ajouté, modifié ou supprimé entre deux versions.

Revenir en arrière

Restaurer un fichier ou le projet à un état antérieur.

Facilite la collaboration

créer des branches pour développer de nouvelles fonctionnalités, corriger des bugs ou tester des idées indépendamment.

COMMENT ÇA FONCTIONNE ?

1. Modifier
Le développeur change les fichiers.

→

2. Enregistrer
Le VCS conserve un point dans l'historique.

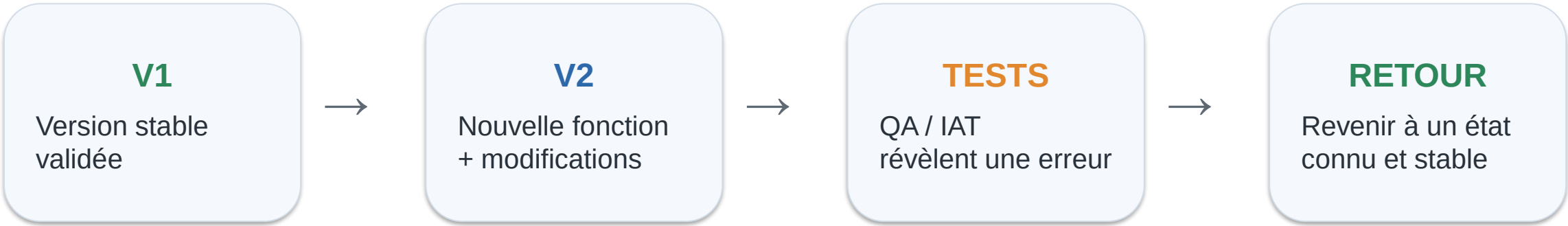
→

3. Comparer / restaurer
L'équipe examine les différences ou retrouve une version antérieure.

Exemples : Git (outil), GitHub / GitLab (plateformes d'hébergement et de collaboration).

Le scénario V1 → V2 : pourquoi l'historique compte

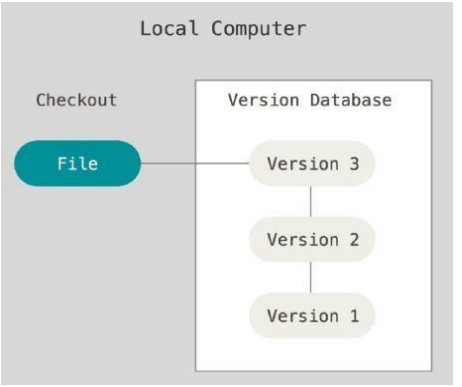
Reprenons le cycle de déploiement présenté avant



Avec VCS (e.g. Git), on peut identifier le changement responsable, comparer V1 et V2, puis restaurer ou corriger le code.

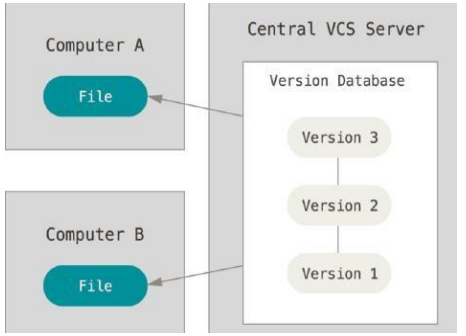
Types de systèmes de contrôle de version

- **Local Version Control:** Simple tracking on one machine

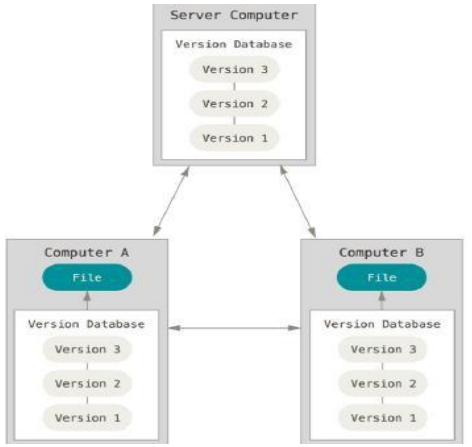


- **Centralized (CVCS):** Single server stores all versions

  (e.g., SVN : Apache Subversion)



- **Distributed (DVCS):** Every user has a full copy (e.g., Git)



Les principales solutions de contrôle de version

1. Systèmes de contrôle de version

- **Git** : système distribué, largement utilisé.
- **SVN (Apache Subversion)** : système centralisé.
- **Perforce (Helix Core)** : adapté aux grands projets.
- **Mercurial (Hg)** : système distribué, alternative à Git.

2. Plateformes de collaboration

- **GitHub** : hébergement de dépôts Git et collaboration.
- **GitLab** : hébergement Git et intégration CI/CD.



dépôt distant (***Remote Repository***) hébergé dans le cloud= Plateforme d'hébergement de dépôts Git

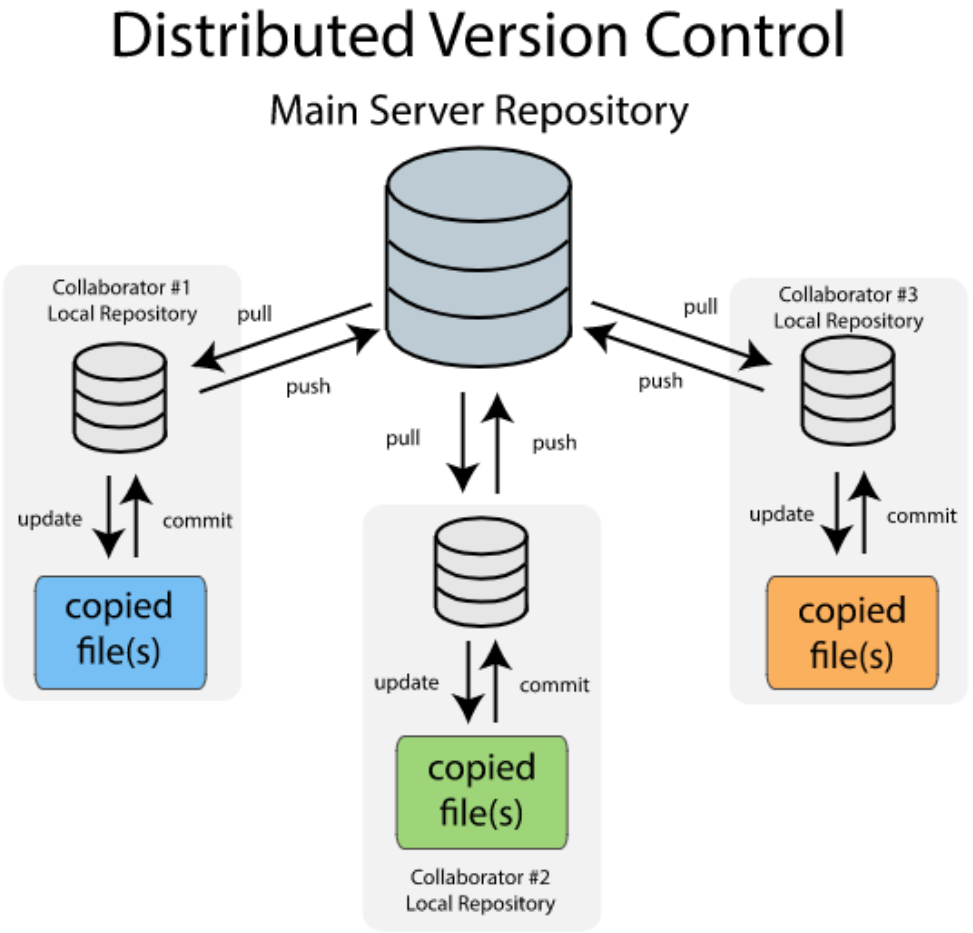
CDO (Connected Data Objects) et **EMF Store** gèrent les versions des modèles

Qu’est-ce que Git ?

- Git est un système de contrôle de version distribué (DVCS).
- Contrairement à un système centralisé, chaque développeur possède une copie du projet pouvant contenir l’historique complet des modifications du code.



Linus Torvalds
Créateur du noyau Linux (Linus + Unix) en 1991,
à l’âge de 21 ans, et créateur de Git en 2005.



Objectifs de Git :

- Rapidité
- Prise en charge du développement non linéaire (des milliers de branches en parallèle)
- Architecture entièrement distribuée
- Gestion efficace des projets de grande taille

Pourquoi GitHub est-il utile ?

- Gestion de versions
- Collaboration
- Documentation
- Reproductibilité
- Visibilité

Histoire de git : du noyau linux à l'open source

1991
Naissance de Linux

Linus Torvalds publie le noyau Linux ; le projet grandit grâce à une communauté de contributeurs.

→

1991–2002
Avant Git

Les changements circulent notamment sous forme de correctifs (patches) et d'archives.

→

2002
BitKeeper

Le projet Linux adopte BitKeeper, un système de contrôle de version distribué propriétaire, utilisé gratuitement à l'époque.

→

2005
Création de Git

Après la fin de l'accès gratuit à BitKeeper, Linus Torvalds et la communauté développent Git pour répondre aux besoins du noyau Linux.

→

Aujourd'hui
Écosystème open source

Git est largement utilisé dans les projets libres et professionnels ; GitHub et GitLab facilitent l'hébergement et la collaboration.

Que permet Git au quotidien ?

COMMIT

Créer un point de sauvegarde nommé dans l'historique.

DIFF

Comparer les modifications entre versions.

BRANCH

Développer une fonctionnalité sans perturber directement la branche principale.

MERGE

Réunir les changements de plusieurs branches.

LOG

Consulter l'historique des commits.

REVERT / RESTORE

Annuler un changement ou récupérer des fichiers selon le besoin.

Le travail en équipe sans écraser le code

Exemple : deux développeurs travaillent en parallèle.

Branche A — Développeur 1

Ajoute la validation du formulaire.

Branche B — Développeur 2

Corrige la gestion des erreurs.



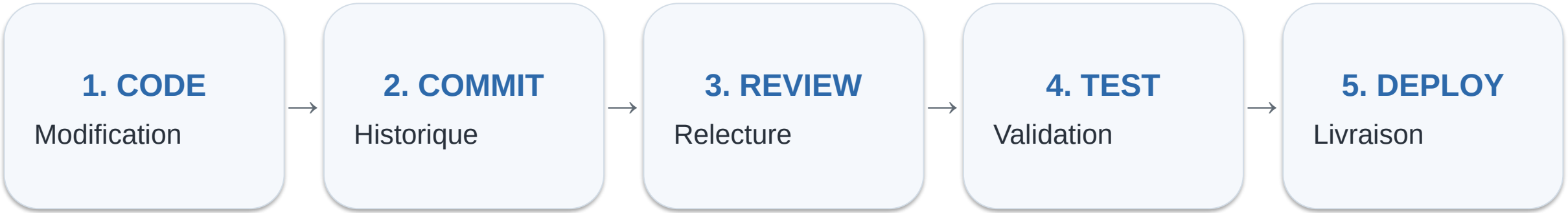
Fusion (merge)

Réunir les changements ; résoudre les conflits si deux modifications se contredisent.

Le VCS facilite la collaboration, mais les conflits doivent être examinés et testés.

Le contrôle de version dans une chaîne DevOps

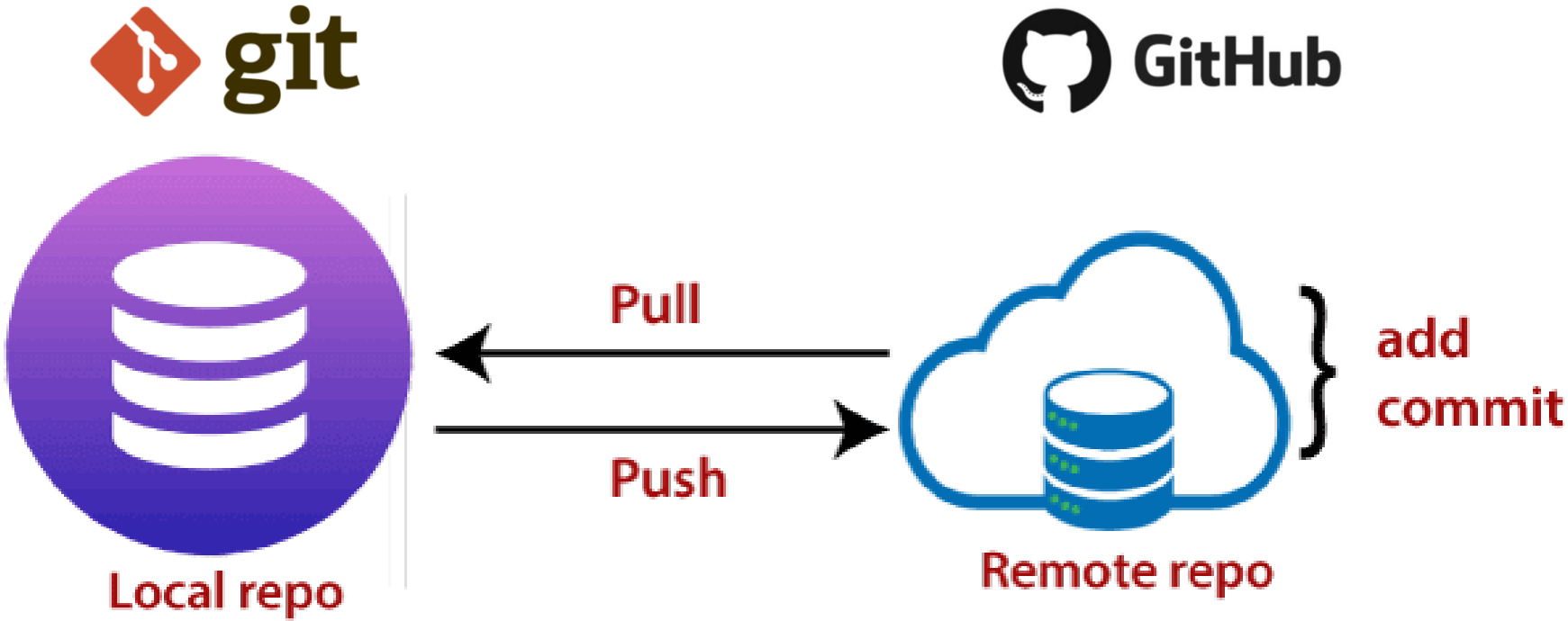
Le VCS aide à rendre les changements traçables et les livraisons reproductibles.



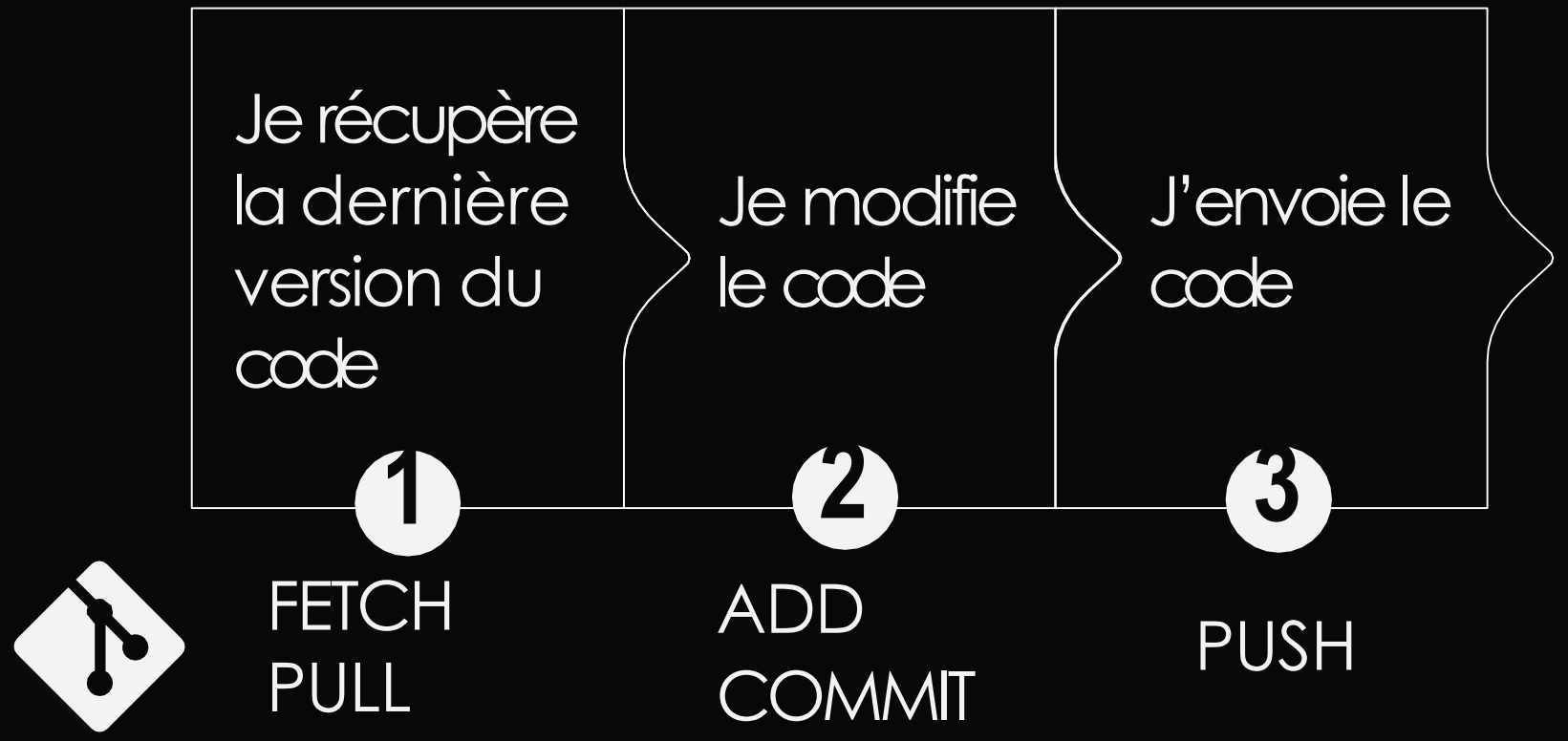
Si les tests échouent, on bloque la livraison, on corrige ou on revient à un état connu.

Le VCS ne remplace ni les tests, ni les sauvegardes, ni la supervision : il complète ces pratiques.

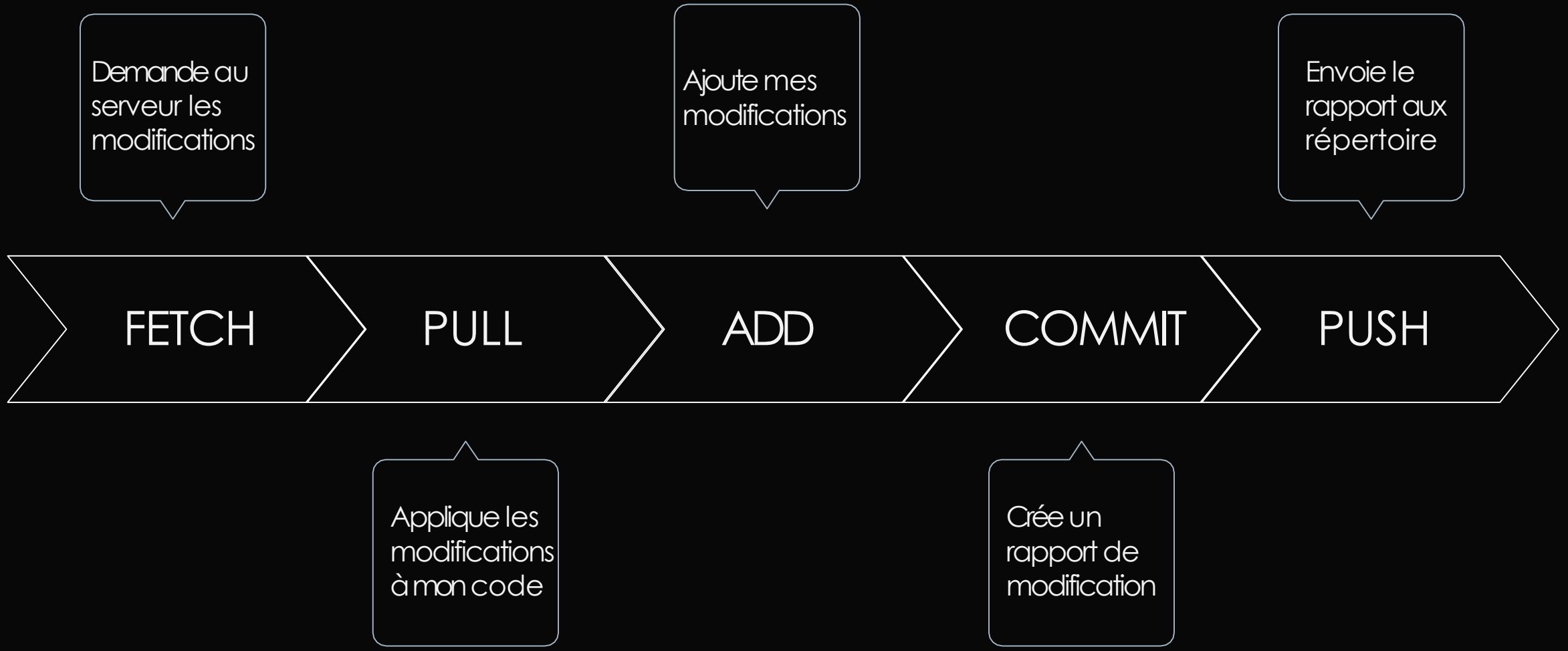
Git/GitHub



EN GROS…


Je récupère la dernière version du code
Je modifie le code
J'envoie le code
1
2
3
FETCH
PULL
ADD
COMMIT
PUSH

LES BASES

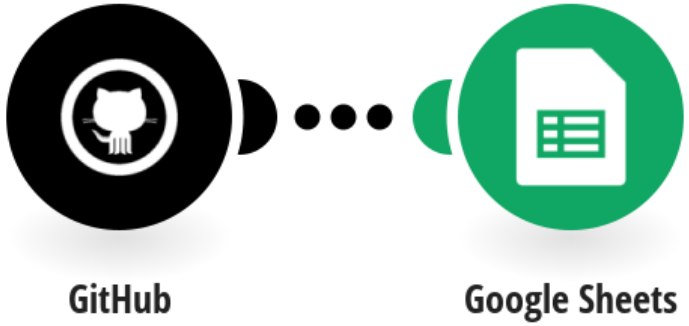

Demande au serveur les modifications
Ajoute mes modifications
Envoie le rapport aux répertoire
FETCH
PULL
ADD
COMMIT
PUSH
Applique les modifications à mon code
Crée un rapport de modification

Le vocabulaire Git à connaître

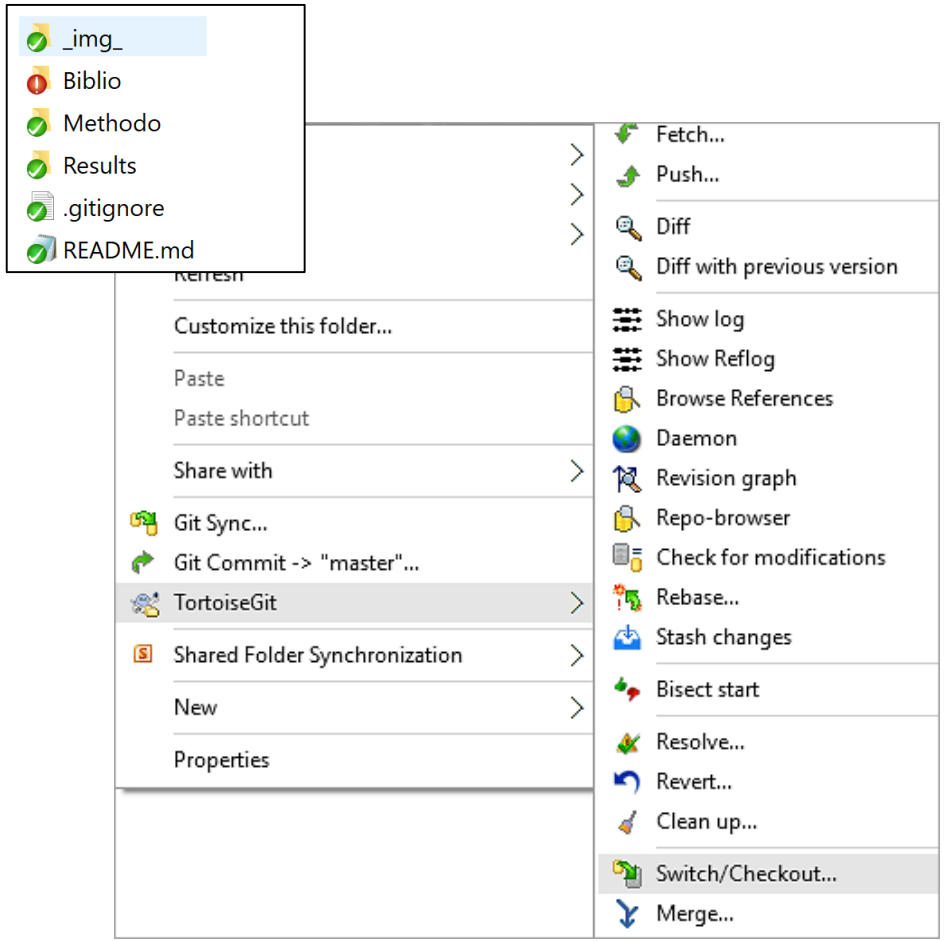
Repository (repo)	Le projet suivi par Git et son historique.	**Clone**	Créer une copie locale d’un dépôt distant.
Commit	Enregistrer un état du projet avec un message.	**Branch**	Développer une fonctionnalité isolément.
Push / Pull	Envoyer / récupérer des commits.	**Merge**	Réunir les changements de plusieurs branches.
Conflict	Deux changements incompatibles sur une même zone.	**Pull Request**	Proposer une contribution pour revue et intégration.

TP N° 2: Outils nécessaires

- **Git/GitHub** -> DevOps Practices
- **GitHub Desktop:** GitKraken, SmartGit, **TortoiseGit**
- **Integrate with IDEs (**IntelliJ IDEA, Eclipse, Visual studio, Overleaf , Matlab)
- Wiki Pages
- **JIRA, Trello**
- **Google Sheet**
  - To Do List

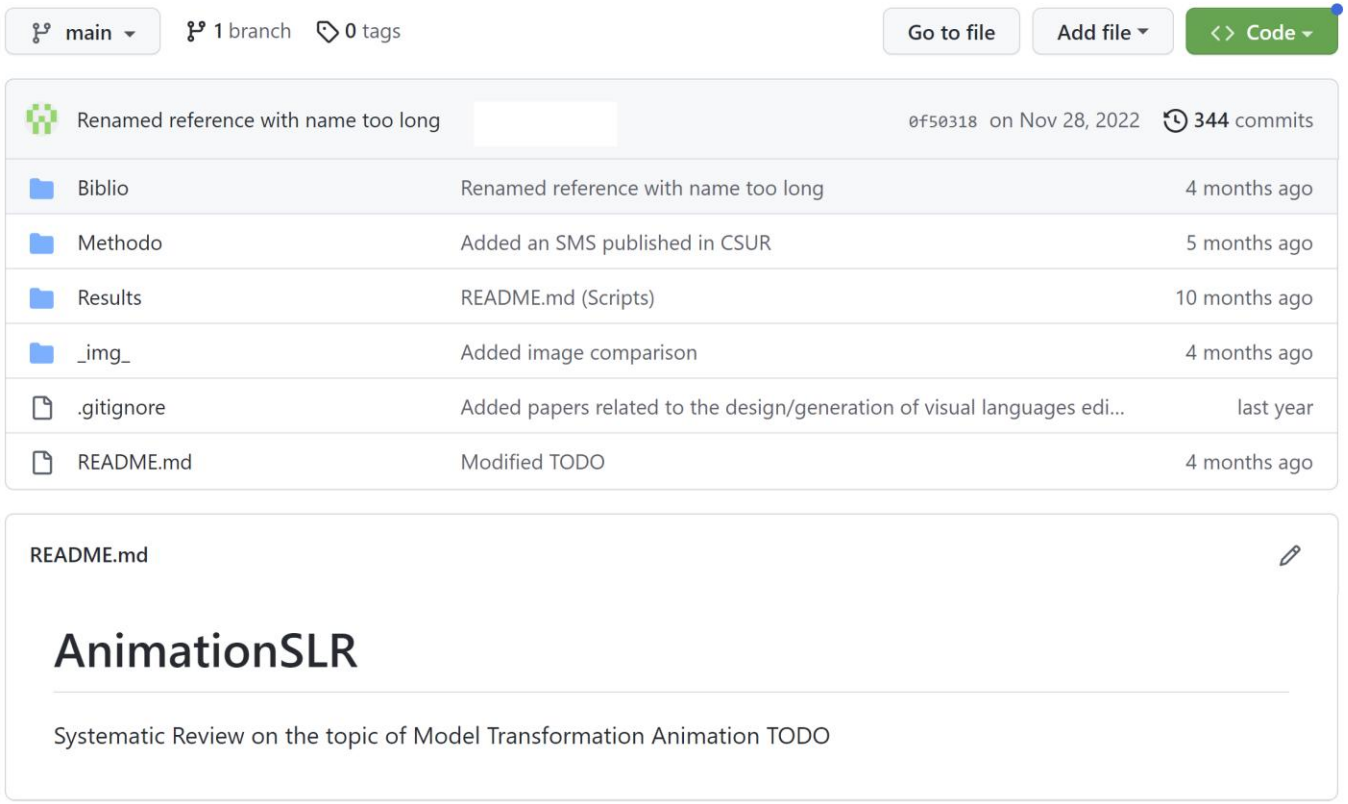


Git and TortoiseGit



git

GitHub



GitHub

Q&A



